Application Proposal

Overview

Application Name: Curated Bites

Description of App: This application is a restaurant discovery & review platform that enables users to find dining options based on their personal preferences, dietary restrictions, and location. The users can then browse through restaurants, read and write reviews, save favorites, and then filter results based off cuisine style, price ranges, dietary accommodations, and ratings from other users.

Target Users:

- People who are looking for restaurant recommendations around them
- People who can share their experiences and leave reviews
- Diners with dietary restrictions to filter out places so they can find food they can eat
- Tourists and locals just being able to explore some new dining options

Core functionality:

- Search and filter restaurants through various criteria such as cuisine type, prices, locations, dietary options
- Read detailed restaurant info such as menu items, hours, contact information, and amenities
- Able to submit and read user reviews with ratings
- The user is able to save favorite restaurants to a list
- Browse cuisine options and dietary preferences
- View combined ratings and/or popular dishes from restaurants

Use Cases

- A user writes a review for a restaurant they had visited
- A user who has dietary restrictions can find a suitable option for dining that has accommodations
- A user can search for a certain cuisine within their budget
- A user can search for restaurants by area
- A restaurant owner or management can update their restaurant information
- Users can view popular dishes at restaurants

Data Requirements

User

- UserID
- Username
- Email
- Password(hashed)
- Account creation date
- Profile picture

Restaurant
- RestaurantID
- Name
- Address
- City
- Phone
- Website
- Description
- Hours
- PriceRange

Reviews
- ReviewID (PK)
- UserID (FK)
- RestaurantID (FK)
- Rating
- Description
- Date
- reviewRatingCounter

Cuisine
- CuisineID (PK)
- Name

Diet
- DietaryOption (PK)
- Name

Menu
- MenuItemID (PK)
- RestaurantID (FK)
- Name
- Description
- Price
- Category

Favorites
- FavoriteID (PK)
- UserID (FK)
- RestaurantID (FK)

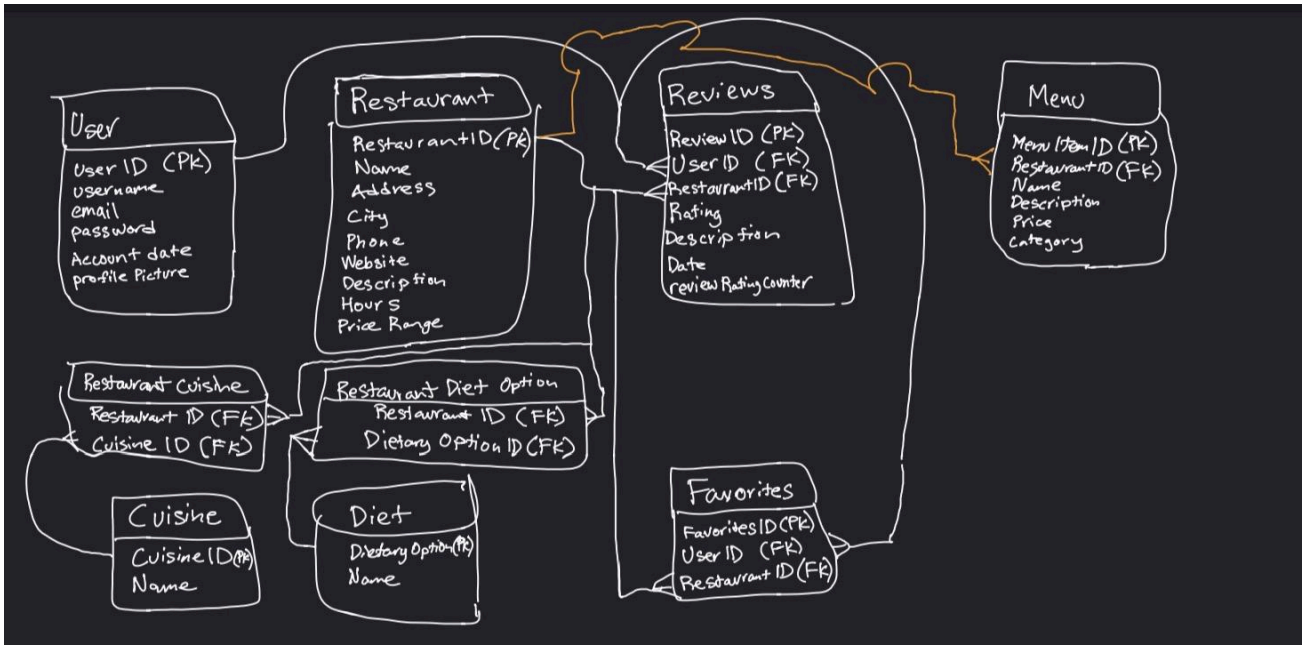
Relationships
User -> Review (1-to-Many)
Restaurant -> Review (1-to-Many)
Restaurant -> Cuisine (Many to many)

Restaurant -> Diet (Many to many)
Restaurant -> Menu (1 to many)
User -> favorite (one to many)
Restaurant -> favorite (one to many)

Rules

- A review can be linked to only one user and one restaurant
- Rating has to be between 1-5 stars
- Items on menu has to be positive value
- A restaurant has to have atleast one cuisine type assigned to it
- The restaurants rating will be calculated by average of all rating reviews
- Restaurant has to have price range between 1 being least expensive and 4 most expensive

ER Diagram



Relational Schema

User (UserID(PK), Username, email, password, googleID(may need if using outh2.0) , creationdate, profilePhoto)
Restaurant (RestaurantID(PK), Name, address, city, phone, website, description, hours, priceRange)
Review (ReviewID(PK), UserID(FK), RestaurantID(FK), Rating, description, date, reviewRatingCounter)

- FK UserID -> PK User (UserID)
- FK RestaurantID -> PK Restaurant (RestaurantID)

Cuisine (CuisineID(PK), Name)
Diet (DietaryOptionID(PK), Name)
Menu (MenuItemID(PK), RestaurantID(FK), Name, Description, Price, Category)
- FK RestaurantID -> PK Restaurant (RestaurantID)

Favorite (FavoriteID(PK), UserID(FK), RestaurantID(FK))
- FK UserID -> PK User (UserID)
- FK RestaurantID -> PK Restaurant (RestaurantID)

Restaurant Cuisine (RestaurantID(PK, FK), Cuisine(PK, FK))
- FK RestaurantID -> PK Restaurant (RestaurantID)
- FK CuisineID -> PK Cuisine (CuisineID)

Restaurant Diet (RestaurantID(PK,FK), DietaryOptionID(PK, FK))
- FK RestaurantID -> PK Restaurant (RestaurantID)
- FK DietaryOptionID -> PK Diet (DietaryOptionID)

Design Justification

My schema supports my use cases since when a user can search for restaurants within their budget, it pulls the price range from the price range in the Restaurant Table, which can allow them to then filter by the price range. And if a user has dietary restrictions, they can filter out places without dietary accommodations from the restaurant diet and diet table that's joined to the restaurant table.

A tradeoff would probably be separating the cuisine and diet into two tables rather than having different values in one table, which has the benefit of ensuring data integrity, although the downside would be that queries may require having multiple joins to get the information. Although I decided to prioritize having data integrity in this case, even though may not necessary, it ensures nothing goes wrong with food types and potential mix-ups with dietary accommodations with different restaurants stored within the database.